CS61A Mock Exam · 参考答案

What Would Python Display · Lists · Trees ｜ 共 3 题，59 分

> 本答案中所有 Python 输出均已在 Python 3 解释器中实际运行验证；
> Q3 的三个函数均通过全部 doctest，并与暴力枚举结果做了 5000 组随机对拍（0 处不一致）。
> 试卷中已给出的示例答案（2022、4 5 / Error、T、F）用灰色标注，其余为解答。

1 What Would Python Display?（14 分）

先给出代码中对象的具体内容：

```
study = [1, 2, 3, 4]           # sum(study) = 10
relax = [1] * 4                # [1, 1, 1, 1], sum = 4
games = [1, 4]                 # sum = 5
meals = [1] + [1] + [1]        # [1, 1, 1], sum = 3, len = 3
todos = [study, relax, games, meals]   # len(todos) = 4
```

stay_up_late 的本质是「每天最多睡 8 小时，逐个 todo 扣时间」：

- ① remain_time < 8 时立刻返回 True（今天熬不动了，但任务还没做完 → 要熬夜）；
- ② 任务全部做完且剩余时间 ≥ 8 时返回 False（不用熬夜）；
- ③ 否则递归处理下一个 todo，并把 hours_per_day 作为剩余时间继续传递。

答案表

表达式	Interactive Output	要点
`pow(2, 11) - 26`	`2022`	2^{11} = 2048，2048 − 26 = 2022（示例）
`print(4, 5) + 1`	`4 5` `Error`	print 先打印 4 5，返回 None；None + 1 → TypeError（示例）

`f1 = print(2+3)`
`f2 = lambda: print(2+3)`
`print(f1, f2, f2())`
	`5`
	`5`
	`None Function`
	`None`
		f1 = print(2+3) 先打印 5 并把 None 绑定给 f1；
		f2 是函数值；求值 f2() 时再打印一个 5 并返回 None

`print(sum(study))`
`print(len(meals))`
`print(len(todos))`
	`10`
	`3`
	`4`
		sum([1,2,3,4]) = 10；meals 有 3 个元素；todos 有 4 个列表

`dropped = True and ((True or False) or True)`
`good_night = dropped or not stay_up_late`
`print(dropped, good_night)`
	`True True`	and/or 短路求值：dropped 为 True 后 `or` 直接返回 True，
		根本不会去求值 `not stay_up_late`

`print(stay_up_late(todos))`
`print(stay_up_late(todos, 48))`
`print(stay_up_late([meals]))`
	`True`
	`False`
	`False`
		24→14→10→5 < 8，返回 True；
		48 一路扣到 remain = 26 且 todos 空 → False；
		[meals] 扣完 24−3 = 21 ≥ 8 且空 → False

表达式	Interactive Output	要点

`tiktok = time_sequence(todos, 0)`
`for todo in tiktok:`
`  if todo >= 3:`
`  print(next(tiktok))`
`  else:`
`  print(todo)`
`alltime = 0`
`for todo in tiktok:`
`  alltime += todo`
`print(alltime)`
	`4`
	`3`
	`0`
		生成器依次产出 10、4、5、3；第一次取到 10 ≥ 3 便用 next 吃掉 4，
		第二次取到 5 ≥ 3 又吃掉 3，此后 StopIteration 结束循环；
		第二个 for 时生成器已耗尽 → alltime 仍为 0

说明 第 3 行中函数值按题目约定写成 Function（真实 REPL 会显示 <function <lambda> at 0x...>）。打印顺序为：先 5（f1 = print(2+3)），再 5（f2() 内部），最后一行 None Function None。

2 Lists（22 分）

2.1 谁能拿到蛋糕（6 分）

执行前两行后，lst 是一个自引用列表：

```
lst = ["trap", [<lambda>, lst]]        # lst[1] 的第 2 个元素就是 lst 自己
```

注意表格里每个表达式都应当以「刚执行完前两行」时的 lst 为初始状态独立求值。

Expression	Your Answer	理由
`(lambda: "cake")()`	T	无参 lambda 直接返回 "cake"（示例）
`(lambda x: "cake")()`	F	缺少参数 → TypeError（示例）
`lst[0]`	F	返回字符串 "trap"，不是 "cake"
`lst[1][0]`	F	返回的是函数值，没有调用它
`lst[1][0]()`	T	取出 lambda 并调用 → "cake"
`lst[1:][0]()`	F	lst[1:] = [lst[1]]，取 [0] 得到列表 lst[1]，列表不可调用 → TypeError
`(lst+[lst])[2][1][0]()`	T	lst+[lst] 的第 3 项是 lst；lst[1] 是 [<lambda>, lst]； 再取 [0] 得 lambda → "cake"
`lst.pop()[1][1][0]()`	F	pop() 先执行并把 lst 改成 ["trap"]； 随后 [1][1] 访问 lst[1] → IndexError

说明 最后一个表达式是本题的陷阱：Python 先求值 lst.pop()（产生副作用），之后链式下标看到的已经是长度为 1 的 lst，因此抛 IndexError，输出 F。

2.2 单个列表推导式（4 分）

l = [[1,2], [3,4], [5,6], [7,8], [9,10]]，题目要求每个答案都是「单个列表推导式」。

目标值	答案
[1, 3, 5]	`[x for x, y in l if y < 8]  # 题目给出的示例`
[[10,9], [8,7], [6,5], [4,3], [2,1]]	`[[y, x] for x, y in l[::-1]]`
[2, 4]	`[y for x, y in l if y < 5]`

说明 第三个答案不唯一，只要能产出 [2, 4] 且形如单个推导式即可，例如 [[y for x, y in l][0], [y for x, y in l][1]] 或 [y for x, y in l if y % 2 == 0 and y < 5]；推荐写法 y < 5 最简洁。

2.3 Diagram #1（6 分）

```
a = [1, 2, [3, 4]]
b = a          # b 与 a 指向同一个列表对象
c = a[:]       # 浅拷贝：c 是新列表，但 a[2] 与 c[2] 共享同一个内层列表
a[1] = b[0]    # a[1] = 1，同时 b 也变（同一对象）
c[2][0] = 5    # 内层列表被改为 [5, 4]，a[2] 与 c[2] 同时改变
```

最终环境图：



a → [1, 1, [5, 4]]　　b 与 a 是同一个对象（b is a 为 True）
c → [1, 2, [5, 4]]　　c 是独立的外层列表，但与 a 共享内层列表

说明 评分要点：a、b 指向同一对象；c 的外层是新列表；a、b、c 三个指针共享同一个内层 [5, 4]。

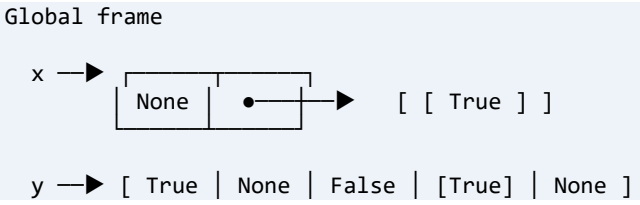
2.4 Diagram #2（6 分）

```
x = [False, True, False]
y = []
for _ in range(5):
    if x[0]:
        x = [x.append([x[0]])] + x[1:]
    else:
        x = x[1:]
    y.append(x[0])
```

逐轮追踪（关键：x.append(...) 返回 None，且它先修改了 x，之后才取 x[1:]）：

轮次	x[0] 真假	本轮之后 x	y 追加	y
1	False	[True, False]	True	[True]
2	True	[None, False, [True]]	None	[True, None]
3	None（假）	[False, [True]]	False	[True, None, False]
4	False	[[True]]	[True]	[True, None, False, [True]]
5	[[True]]（真）	[None, [[True]]]	None	[True, None, False, [True], None]

最终环境图：

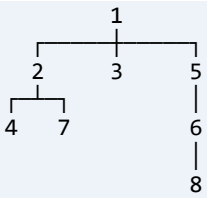



```
x = [None, [[True]]]
y = [True, None, False, [True], None]
```

说明 第 2 轮最易错：x.append([x[0]]) 先把 x 改成 [True, False, [True]] 并返回 None，因此 [x.append(...)] 是 [None]，此后再取 x[1:] 得到 [False, [True]]。

3 Trees（23 分）

由题目中的 doctest 反推树结构：sum_leaves(t1) = 4+7+3+8 = 22、depth(t1,8) = 3、max_path(t1,4) = 1+5+6+8，且 branches(t1)[0] 的叶子是 4 和 7，故 t1 为：



3.1 用 tree 构造 t1（3 分）

```
t1 = tree(1, [tree(2, [tree(4), tree(7)]),
              tree(3),
              tree(5, [tree(6, [tree(8)])])])
```

说明 三个子树从左到右依次为 2、3、5 才能与 branches(t1)[0] 的 doctest 对上。

3.2 sum_leaves（3 分）

```
def sum_leaves(t):
    """ Returns sum of labels of t's leaves.
    >>> sum_leaves(t1) # 4 + 7 + 3 + 8
    22
    """
    if is_leaf(t):
        return label(t)
    return sum([sum_leaves(b) for b in branches(t)])
```

说明 空白依次为：is_leaf(t) / label(t) / sum_leaves(b) / b / branches(t)。

3.3 depth（6 分）

```
def depth(t, v):
    """ Returns the depth of the node with given label, or None if not found.
    >>> depth(t1, 1), depth(t1, 3), depth(t1, 8), depth(t1, 9)
    (0, 1, 3, None)
    """
    if label(t) == v:
        return 0
    for b in branches(t):
        d = depth(b, v)
        if d is not None:
            return d + 1
    return None
```

行	空白处应填
6	`label(t) == v`
7	`0`
8	`for b in branches(t):`
9	`depth(b, v)`
11	`d + 1`
12	`None`

说明 第 10 行的 `if d is not None:` 已经给出，它是本题的关键：用 None 表示「没找到」，从而 0 这个合法深度不会被当成假值。

3.4 sum_lowest_leaves（4 分）

思路：用 helper 维护「同一层的所有子树」列表；当它们全是叶子时，本层就是最深层，直接求和；否则把所有子树的 branches 拼成下一层，继续下潜。

```
def sum_lowest_leaves(t):
    """ Returns the sum of labels of t's lowest leaves.
    >>> sum_lowest_leaves(t1)
    8
    >>> sum_lowest_leaves(branches(t1)[0]) # 4 + 7
    11
    >>> sum_lowest_leaves(tree(1, [tree(2), tree(3), tree(4)]))
    9
    """
    def helper(trees):
        if all([is_leaf(x) for x in trees]):
            return sum([label(x) for x in trees])
        return helper(sum([branches(x) for x in trees], []))
    return helper([t])
```

行	空白处应填
11	`all([is_leaf(x) for x in trees])`
12	`sum([label(x) for x in trees])`
13	`sum([branches(x) for x in trees], [])`
14	`[t]`

说明 sum(list_of_lists, []) 是「把若干列表拼接成一个列表」的惯用写法（sum 的 start 参数为 []）。若写成 sum([branches(x) for x in trees], []) 之外的写法，务必保证拼接结果是一个列表。

3.5 max_path（7 分）

思路：helper(t, k, flag) 中 flag 表示「当前路径必须包含 t」（即路径已经延伸到 t）。于是：

- k == 0：没有节点可用了，返回 0；
- t 是叶子，或 flag 为真且 k == 1：路径只能就地停在 t，返回 label(t)；
- max1 =「从 t 开始」的最优路径（至少含 t 本身）；
- 若 flag 为真，路径必须含 t，max1 即答案；否则还要比较 max2 =「从 t 的某个后代开始」的最优路径。

```
def max_path(root, k):
    """ Returns the highest label sum of paths with length no more than k.
    >>> max_path(t1, 2) # 6 + 8
    14
    >>> max_path(t1, 4) # 1 + 5 + 6 + 8
    20
    >>> max_path(tree(1, [tree(10, [tree(20, [tree(5)])])]), 2)
    30
    >>> max_path(tree(5, [tree(-1, [tree(-2, [tree(-3)])])]), 2)
    5
    """
    def helper(t, k, flag):
        if k == 0:
            return 0
        elif is_leaf(t) or (flag and k == 1):
            return label(t)
        max1 = max([label(t)] + [label(t) + helper(b, k - 1, True)
                                 for b in branches(t)])
        if flag:
            return max1
        max2 = max([helper(b, k, False) for b in branches(t)])
        return max(max1, max2)
    return helper(root, k, False)
```

行	空白处应填
13	`k == 0`
15	`is_leaf(t) or (flag and k == 1)`
17	`max([label(t)] + [label(t) + helper(b, k - 1, True) for b in branches(t)])`
18	`flag`
19	`return max1`
20	`max([helper(b, k, False) for b in branches(t)])`
21	`max(max1, max2)`

验证结果

断言	运行结果	期望
`sum_leaves(t1)`	22	22 ✓
`depth(t1,1), depth(t1,3), depth(t1,8), depth(t1,9)`	(0, 1, 3, None)	(0, 1, 3, None) ✓
`sum_lowest_leaves(t1)`	8	8 ✓
`sum_lowest_leaves(branches(t1)[0])`	11	11 ✓
`sum_lowest_leaves(tree(1, [tree(2),tree(3),tree(4)]))`	9	9 ✓
`max_path(t1, 2)`	14	14 ✓
`max_path(t1, 4)`	20	20 ✓
`max_path(tree(1,[tree(10,[tree(20, [tree(5)])])]), 2)`	30	30 ✓
`max_path(tree(5,[tree(-1,[tree(-2, [tree(-3)])])]), 2)`	5	5 ✓
与暴力枚举对拍（随机树 5000 组，k∈[1,4]）	0 处不一致	—

说明 第 15 行必须带上 flag 条件（不能只写 k == 1）：当 flag 为假时，即使只剩 1 个节点可用，路径也允许从子树中的任意节点开始，直接返回 label(t) 会漏掉更优的解。另外此处不允许「空路径」，因此 t 为叶子时一律返回 label(t)，这正好符合「标签可为负」的设定。